

Ejercicios de repaso

Cálculo de costes

1. ¿Cuál es el coste en tiempo y en espacio adicional de las siguientes funciones?

```
int desorden(const vector<double>& v)
{
    const int N = v.size();
    int cambiados = 0;

    for (int i = 0; i < N; ++i)
        for (int j = i + 1; j < N; ++j)
            if (v[i] > v[j])
                cambiados++;

    return cambiados;
}
```

El coste de `desorden` es $O(N^2)$ en tiempo y $O(1)$ en espacio adicional. El bucle interior da $N-i-1$ iteraciones y el bucle exterior da N . Sumando $\sum_{k=1}^{N-1} k$ se obtiene $O(N^2)$. El único espacio adicional son las variables locales enteras de la función.

```
int suma(vector<int> v, int desde)
{
    if (desde >= v.size()) return 0;
    else return v[desde] + suma(v, desde + 1);
}
```

El coste de `suma` es $O(N^2)$ en tiempo y en espacio adicional por la copia innecesaria de `v`, que cuesta y ocupa $O(N)$ en cada iteración. Sin embargo, cambiando el argumento por `const vector<int>& v`, los costes en tiempo y en espacio pasan a ser $O(N)$.

```
char letraMasFrecuente(const vector<string>& texto)
{
    map<char, int> veces;

    for (const string& palabra : texto)
        for (char c : palabra)
            veces[c] += 1;

    char masFrecuente = 0;
    int masVeces = 0;

    for (auto [letra, f] : veces)
        if (f > masVeces) {
            masFrecuente = letra;
            masVeces = f;
        }

    return masFrecuente;
}
```

En la función frecuencias, el número de palabras del texto es N , el número de letras del texto es M y el número de letras distintas en el texto es D .

El coste en tiempo de frecuencias es $O(M \log D + N) = O(M \log D)$ por M inserciones en un árbol de búsqueda equilibrado de tamaño a lo sumo D . Si suponemos que todas las palabras tienen al menos una letra, $M \geq N$ y se puede simplificar (si no, no). El coste en espacio adicional es $O(D)$, pues es lo que ocupa la variable veces.

2. Si solo sabemos que $0 \leq f(n) < n$ para cualquier $n > 0$ y $f \in O(n^2)$, ¿cuál es el coste en tiempo de la siguiente función?

```
int foo(int n)
{
    int suma = 0, x = n;

    while (x > 0) {
        suma += x;
        x = f(x);
    }

    return suma;
}
```

El bucle se ejecuta como mucho n veces (si $f(x) = x - 1$) y como mucho tarda $\sum_{k=1}^n O(k^2 + 1) = O(n^3)$.

Diseño de algoritmos recursivos

En estos ejercicios, escribe ecuaciones recursivas para resolver el problema, no necesariamente el código en C++.

3. Supongamos que tenemos un lenguaje de programación donde el único operador aritmético es la suma, ¿cómo podrías definir una función $\text{mult}(m, n)$ que calcule el producto de m por n ?

$$\begin{aligned} \text{mult}(m, 0) &= 0 \\ \text{mult}(m, n) &= \text{mult}(m, n - 1) + m && \text{si } n \geq 1 \end{aligned}$$

4. La sucesión de Fibonacci se puede definir con las ecuaciones $\text{fib}(0) = 0$, $\text{fib}(1) = 1$ y $\text{fib}(n) = \text{fib}(n - 1) + \text{fib}(n - 2)$ si $n \geq 2$. El coste de evaluar esta función no es lineal en n . Si la función pudiera devolver más de un valor entero, ¿cómo podrías definir un $\text{fibln}(n) = \text{fib}(n)$ con coste lineal?

$$\begin{aligned} \text{fibln}(0) &= (0, 0) \\ \text{fibln}(1) &= (1, 0) \\ \text{fibln}(n) &= (f_1 + f_2, f_2) && \text{si } n \geq 2 \wedge (f_1, f_2) = \text{fibln}(n - 1) \end{aligned}$$

5. Los números de Catalan se pueden calcular recursivamente como

$$C_0 = 1 \quad \text{y} \quad C_{n+1} = \frac{2(2n+1)C_n}{n+2}.$$

Escribe una función $\text{cat}(m)$ que calcule el primer número de Catalan mayor que m . ¿Y si se pide el índice n de dicho valor? Generaliza los argumentos como consideres necesario.

$$\begin{aligned} \text{cata}(m) &= \text{cata}(m, 0, 1) \\ \text{cata}(m, n, c) &= \begin{cases} c & \text{si } c \geq m \\ \text{cata}\left(m, n+1, \frac{2(2n+1)c}{n+2}\right) & \text{si } c < m \end{cases} \end{aligned}$$

6. Este año se celebra un gran festival de música en el jardín botánico de la universidad. Cada noche actúa un grupo distinto y has elaborado una lista con aquellos a los que te gustaría escuchar junto con una puntuación numérica v_k y el coste de su entrada c_k . Además, has ahorrado una cantidad C para comprar las entradas, que no te importaría gastar al completo. Sin embargo, el dinero ahorrado no da para asistir a todos los conciertos que te gustan, así que habrá que elegir. ¿Cómo puedes hacerlo de tal forma que la elección tenga la mayor puntuación posible y el coste de las entradas no se pase del presupuesto?

Sean n el número de conciertos, $v_1, \dots, v_n$ sus puntuaciones asignadas, $c_1, \dots, c_n$ los precios de sus entradas y C el presupuesto total. Entonces,

$$\begin{aligned} \text{pmax}(0, c) &= 0 \\ \text{pmax}(k, c) &= \max \{ \text{pmax}(k-1, c - c_k) + p_k, \text{pmax}(k-1, c) \} & \text{si } c_k \leq c \\ \text{pmax}(k, c) &= \text{pmax}(k-1, c) & \text{si } c_k > c \end{aligned}$$

7. Escribe una función recursiva que calcule de cuántas formas distintas se puede obtener la cantidad S sumando en orden todos o una parte de n números enteros positivos dados. Por ejemplo, con los números 3, 5, 4, 4 y 1, hay 3 formas distintas de sumar 8 ($3 + 5$, $4 + 4$ y $3 + 4 + 1$).

La función $\text{sumas}(v_1 \dots v_n, s)$ calcula cuántas formas hay de obtener s sumando algunos de los números v_1 a v_n .

$$\begin{aligned} \text{sumas}(w, 0) &= 1 \\ \text{sumas}(\{\}, s) &= 0 & \text{si } s \neq 0 \\ \text{sumas}(vw, s) &= \text{sumas}(w, s) & \text{si } v > s \\ \text{sumas}(vw, s) &= \text{sumas}(w, s - v) + \text{sumas}(w, s) & \text{si } v \leq s \end{aligned}$$

donde $\{\}$ es la lista vacía, v es un número y w es el resto de números.

C++ y demás

8. Escribe una función que calcule el mínimo de una lista no vacía de elementos de un tipo genérico T .

```
template <typename T>
int min(const vector<T>& v)
{
    T minValor = v[0];

    for (int i = 1; i < v.size(); ++i)
        if (v[i] < minValor)
            minValor = v[i];

    return minValor;
}
```

9. ¿Qué tipos de C++ deberían ser `num_t1` y `num_t2` para que no se produzca desbordamiento?

```

num_t1 total = 0;

for (num_t2 v : valores)
    total += v * v;

```

Escoge los más pequeños posibles suponiendo que

1. el tamaño de valores no es mayor que 10^6 y los valores son a lo sumo 1000.
2. el tamaño de valores no es mayor que 1000 y los valores son a lo sumo 10^6 .

El tipo num_t1 debe ser un entero de 64 bits en ambos casos (**int64_t**, **long int** o **long long int** en Windows), pues el resultado puede llegar a ser del orden de $10^{12} \geq 2^{39}$. En el segundo caso, además, num_t2 debe ser también entero de 64 bits. En otro caso, puede ser entero de 32 bits (**int**, **int32_t**).

10. Completa el coste de las siguientes estructuras de datos:

Árbol de búsqueda equilibrado (map)	
Inserción	
Consulta	
Borrado	
Número de elementos	

Tabla hash (unordered_map)	
Inserción	
Consulta	
Borrado	
Número de elementos	

Pila (stack)	
top	
push	
pop	

Cola (queue)	
first	
push	
pop	